Take-home: Metered API Billing

You're joining a team that just shipped the first version of a SaaS API and now needs to bill customers for usage. Build the core of the metering + billing system: ingest usage events, aggregate them into time windows, generate invoices, and expose two front-ends, one for customers, one for internal ops.

There's no hard time cap, but the task is sized for **~4 days with AI assistance**. Don't build more than the brief asks; we'd rather see careful trade-offs than feature breadth. AI use is encouraged.

Deliverables

1. **A working system** in a git repo, runnable locally with `docker compose up`. Include a seed/generator script that produces realistic data.
2. **`DESIGN.md`** — your written reasoning. Counts as much as the code (see rubric).

The system

Domain

- A *customer* signs up and gets one or more *API keys*.
- Every API call from the real product emits a *usage event* (request ID, customer ID, api key ID, endpoint, units consumed, timestamp). For this exercise, simulate the product with a generator script.
- A scheduled job rolls usage events into *usage windows* (one row per customer × hour).
- A scheduled job rolls usage windows into *invoice line items* against the customer's *price plan* (tiered, e.g. first 10k units free, next 90k at $0.001, beyond that at $0.0005).
- Invoices are issued monthly. A mock *payment-processor webhook* marks them paid.
- Ops can: list customers, view a customer's usage and invoices, issue a credit, override a line item, and see basic anomaly signals (e.g. usage 10× a customer's 30-day average).
- Each usage event contains a globally unique request_id. Re-delivery of the same request_id must not produce duplicate billing effects, even if received multiple times or concurrently.
- Usage events may arrive late or out of order. You do not need to implement a full reconciliation system, but your design should explicitly describe how late-arriving events would be handled after aggregation or invoice issuance.

Production target you're designing for

> 5,000 active customers · 200 events/sec sustained, 2,000/sec peak · ~500M events/month · monthly invoices, accuracy is contractual.

You do not need to implement distributed infrastructure or hyperscale architecture. We prefer a simpler system with strong correctness guarantees and a clear evolutionary scaling path over prematurely distributed designs. Explain where your chosen design would break first and what migration path you would take.

APIs to build (minimum)

Customer-facing

- `POST /v1/events` — batched ingestion. Idempotent.
- `GET /v1/usage` — paginated, filterable by date range and api key.

- `GET /v1/invoices`, `GET /v1/invoices/{id}`.

Ops-facing

- `GET /ops/customers`, `GET /ops/customers/{id}`.
- `POST /ops/customers/{id}/credits`.
- `PATCH /ops/invoices/{id}/line-items/{id}` — override with audit trail.
- `POST /webhooks/payments` — signed; verify and handle replays.

Security & isolation requirements

- Every `/v1` endpoint must scope to the authenticated customer. Demonstrate how you prevent a customer from reading another customer's invoice or usage by guessing an ID. Tenant scoping should live somewhere it can't be forgotten, not in each view.
- API keys must not be retrievable in plaintext after creation. Show how you store and verify them.
- The webhook endpoint must verify a signature against a shared secret loaded from the environment, and must be safe under replay (same delivery received twice ≠ double-effect).
- Audit log entries for credit issuance and line-item override must be immutable and capture actor, timestamp, before/after values, and a reason. Mutating or deleting an audit row should not be possible through normal application code paths.
- No secrets in the repo. Webhook signing key, DB creds, anything similar, env-based.

Front-ends

- *Customer dashboard*: current-period usage with a chart, invoice list, and invoice detail.
- *Ops console*: customer list, customer detail (usage + invoices), credit issuance, line-item override.

These can live in one repo as one or two SPAs. We are evaluating operational UX clarity and safety, not frontend polish. Minimal, functional interfaces are preferred over visually elaborate implementations.

Stack

Pick what you're fastest in. Our stack is Django + DRF, Postgres, React + Vite + TypeScript, AWS — but you're free to choose. Whatever you pick, you'll be asked about the choice. We do expect:

- A real relational database. Money stored in integer minor units.
- Prioritize tests around correctness boundaries: idempotency, concurrency, tenant isolation, reconciliation behavior, and money-moving actions. We are not evaluating trivial coverage metrics.
- A background job mechanism. A simple cron + a locked job table is fine; so is Celery / RQ / Sidekiq.

What `DESIGN.md` must cover

Keep it tight **~1,500–2,500 words**.

1. **Data model.** Schema, indexes, why those indexes, what you'd add at 10× and 100×.
2. **Idempotency & concurrency.** What happens if event ingestion is replayed, the aggregator runs twice, the webhook is delivered three times, ops clicks "issue credit" twice. Show the locking/dedupe strategy.

3. **Aggregation pipeline.** Events → windows → line items. Where state lives, what's recomputable vs. immutable, how you'd reconcile drift between raw events and window totals.
4. **Failure modes.** Three things that break first at production scale, with the fix you'd reach for.
5. **Threat model.** A hostile customer, a hostile internal user, and a compromised webhook source. For each: what's the worst they can do, and what stops them? Be specific about authz scoping, API key storage, and audit trail integrity. Focus on concrete abuse scenarios specific to this system: cross-tenant access, replay attacks, operator misuse, invoice tampering, credential leakage, and duplicate financial actions
6. **Trade-offs.** For at least two non-obvious decisions: what you chose, one alternative you rejected, and why.
7. **What you didn't build and would build next.**

Rubric (how we score)

Eight categories. Levels: *weak / solid / strong / outstanding*.

Category	Weight	What "strong" looks like
Data model & integrity	18%	Right keys & FK behavior, integer money, indexes match the queries you actually run, constraints over comments
Concurrency & correctness	18%	Aggregator and webhook are provably idempotent; concurrent ops actions can't double-credit; ingestion handles replays
Scaling reasoning (writeup)	13%	Specific numbers; identifies *what breaks first*; distinguishes "won't scale" from "scales with a known fix"
API & frontend craft	13%	Sane REST shape; pagination chosen with reasoning; money-moving UI has confirmation + idempotency token; loading/error states aren't an afterthought
Security & isolation	10%	Tenant scoping enforced at the right layer (not in views); API keys handled like secrets; audit entries are immutable; threat model in writeup is specific, not generic OWASP-ese
Trade-off writeup	10%	Considers alternatives; honest about what you'd do differently
Operational thinking	10%	Observability hooks (what would you alert on?); migration story; how ops debugs a wrong invoice
Code quality & testing	8%	Readable; tests cover the bits that would break in production, not getters

Submission

Send the repo URL and any setup notes.
